ENTERPRISE RESOURCE PLANNING

In large corporations *enterprise resource planning (ERP)* systems like SAP contain critical master data about assets, maintenance schedules, procurement records, and organizational hierarchies. This data provides essential context for interpreting operational measurements —knowing that a pump was installed last month explains why its vibration patterns differ from historical baselines. ERP data typically changes slowly and benefits from in-place access through well-established APIs and database connections. Listing 4.2 shows how you can query SAP via an ODATA API to get back sales data. To try this out yourself, you can sign up at https://api.sap.com/ and get an API key that you can use on the publicly available SAP API sandbox.

Listing 4.2 An example of accessing data from a public sandbox of SAP, a common ERP system, to retrieve sales orders.

```
import requests

url = "https://sandbox.api.sap.com/s4hanacloud/sap/opu/
➥odata/sap/API_SALES_ORDER_SRV/A_SalesOrder" #1
headers = {
  "APIKey": "your_api_key_here", #2
  "Accept": "application/json"
}

params = {
  "$top": "5",
  "$select": "SalesOrder,SoldToParty,SalesOrderDate,TotalNetAmount",
}

response = requests.get(url, headers=headers, params=params)

if response.status_code == 200:
  orders = response.json()["d"]["results"]
  print(f"{len(orders)} Sales Orders from SAP:")

  for order in orders:
    so_id = order.get("SalesOrder", "N/A")
    customer = order.get("SoldToParty", "N/A")
    amount = order.get("TotalNetAmount", "N/A")
    print(f"  {so_id}: Customer {customer} - ${amount}")
else:
  print(f"Error: {response.status_code}")
```

#1 Use the free SAP sandbox API and target the sales order API.
#2 Specify your API key as a header.